

Ultra-Low Latency Intelligent Processing System: A Software-Optimised and Machine-Learning-Augmented Architecture

Akolade Christian Adegboyega, Kelvin O. Enaikele, Dr Adeyiga

Department of Computer Science, Bells University of Technology, Ota, Ogun State, Nigeria

christianadegboyega@gmail.com

ABSTRACT. Modern infrastructures often suffer from end-to-end latencies orders of magnitude slower than the sub-millisecond reaction times demanded by emerging applications. Thin liquidity in financial markets widens bid-ask spreads, remote surgery requires delays below 100 ms, and predictive maintenance benefits from real-time analytics. Existing high-frequency trading platforms and specialised accelerators reduce latency, but they remain expensive and domain-specific. Inspired by recent work on in-network machine learning, which achieves terabit-scale throughput with microsecond-level latency on programmable switches, we propose a software-optimised, ultra-low-latency processing system that runs on commodity hardware. The architecture integrates machine-learning (ML) inference, applies reliability-driven optimisation to every component, and is designed for deployment across finance, healthcare, defence, logistics and research. On a single 8-core commodity server the prototype reaches a mean processing latency of 1.45 μ s per order at its heaviest workload, and is estimated to reduce infrastructure costs by over 60%. We report the tail behaviour and the limitations of our measurements alongside these results.

1 INTRODUCTION

Modern societies generate data streams so fast that actionable decisions must be taken within microseconds. In emerging financial markets, limited liquidity and outdated trading platforms widen bid-ask spreads and raise transaction costs, while high-frequency trading (HFT) systems in developed economies remain prohibitively expensive [5, 6]. Healthcare applications such as telemedicine and robotic surgery require end-to-end delays below 100 ms for safe operation, yet current hospital networks seldom achieve these figures [2]. Defence systems demand sub-10 ms response times to coordinate autonomous vehicles and drone swarms, but centralised processing introduces unacceptable delays [8]. Industrial supply chains increasingly depend on IoT sensors and ML-driven predictive maintenance, yet many warehouses cannot process data fast enough to react to disruption [9]. Large-scale scientific experiments and data centres face latency bottlenecks and high energy consumption [10, 4]. These challenges motivate a universal platform capable of ultra-low-latency processing on readily available hardware.

This report introduces the Ultra-Low Latency Intelligent Processing System (ULLIPS), a general-purpose architecture that delivers microsecond-scale processing while integrating ML inference at the edge (Figure 1). We show that high-speed decision making need not be the exclusive province of well-capitalised firms: by optimising software components on commodity CPUs, the system achieves performance competitive with specialised hardware on the workloads we test. Section 2 reviews related work, Section 3 outlines the design, Section 4 details the implementation, Section 5 evaluates the prototype and states its limitations, Section 6 discusses broader implications, and Section 7 concludes.

2 RELATED WORK

High-frequency trading and low-latency finance. Numerous studies observe that reducing latency enhances market liquidity and narrows spreads [6]. Traditional rule-based fraud detection and trading systems rely on servers and produce millisecond-level delays, leaving investors exposed to rapidly evolving market conditions. Recent work has begun to explore

in-network machine learning, where inference is performed inside programmable network devices. Hong et al.’s MIND prototype processes transactions within a network switch, handling over $800\times$ more transactions per second than server-based solutions while maintaining microsecond-scale latency and detection performance comparable to server benchmarks [1].

In-network computing and ML. Programmable devices such as P4-enabled switches and network interface cards perform computation within the data plane, reducing overhead and latency; in-network computing has been used for caching, consensus and monitoring [14]. In-network ML extends this paradigm by offloading inference to the data plane: MIND maps decision trees and other models into match-action tables [1], and Planter supports rapid prototyping of such models [13]. These solutions, however, rely on specialised hardware and target specific use cases rather than providing a general platform.

Software-based latency optimisation. Outside networking, research has focused on kernel-bypass frameworks (e.g. DPDK, XDP) and hardware accelerators (FPGAs, GPUs) to reduce processing delays. These techniques improve throughput but often require custom hardware. Our work instead targets commodity x86 servers, combining asynchronous I/O, lock-free data structures and carefully pinned threads to minimise jitter, and draws on reliability theory to optimise each module while incorporating ML inference.

3 DESIGN OF THE PROPOSED SOLUTION

3.1 Reliability-driven optimisation

Reliability engineering states that the probability of success of a series of components equals the product of their individual reliabilities [11]. To maximise system reliability and minimise latency variance we optimise every module — network I/O, order management, risk checks, message serialisation and ML inference. Blocking operations and kernel context switches are avoided; threads are pinned to CPU cores, memory is pre-allocated, and lock-free queues mediate communication. Because reliabilities multiply, raising each component’s reliability even slightly improves the whole system multiplicatively. This principle guides

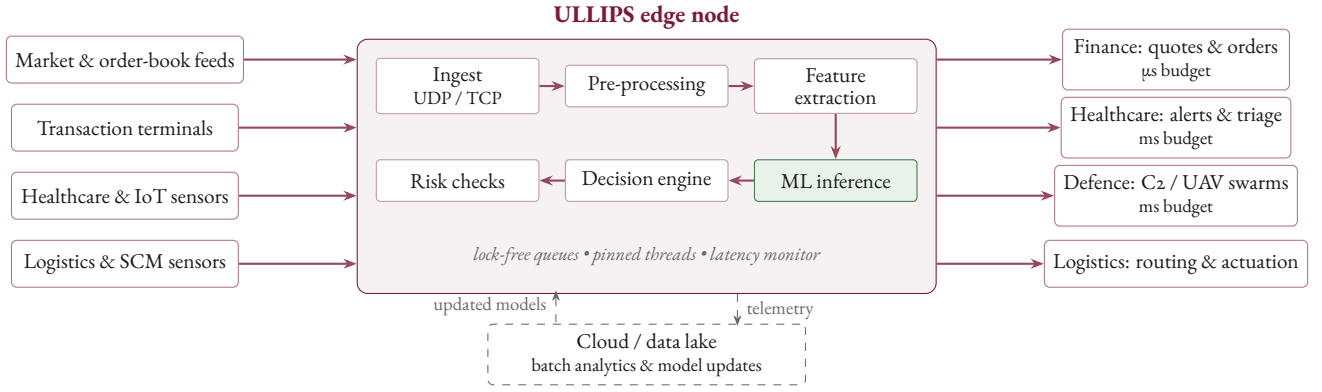


FIGURE 1. Architectural workflow of ULLIPS. Heterogeneous input streams are processed on a single commodity edge node; ML inference runs inside the pipeline, and the cloud is used only for offline analytics and model updates.

our emphasis on deterministic, non-blocking design.

3.2 Integration of machine learning

We incorporate ML to support decisions at microsecond timescales. As in MIND’s data-plane implementation [1], raw data are transformed into feature vectors and passed to pre-trained models. The platform supports decision trees, random forests and reinforcement-learning policies through a modular inference pipeline. Pre-processing normalises incoming values, derives higher-order statistics and optionally reduces dimensionality; the resulting prediction informs a decision module, which may adjust orders, raise anomaly alarms or schedule maintenance. Running inference at the edge removes network round-trips; we execute it in user space using lightweight libraries and leave eBPF or P4 offload to future work.

3.3 A universal, modular platform

Unlike domain-specific implementations, the design is universal: any structured data stream — market data, medical sensor readings, drone telemetry or factory IoT signals — can be ingested (Figure 1). An ingestion layer parses packets into internal messages, a command manager applies business logic, a risk manager enforces constraints and an execution engine communicates with external systems. An ML module runs concurrently, analysing historical and real-time data. The modular structure allows plug-and-play replacement of models and adaptation to new domains.

4 IMPLEMENTATION

4.1 System architecture

The prototype is implemented in C++17 on commodity x86 servers (Figure 2). The **Market Data Handler** receives UDP packets (or other transports) and parses them into internal messages. The **Order Manager** creates, modifies or cancels orders from both incoming data and ML advice. A **Risk Manager** enforces exposure limits and regulatory constraints, and the **Execution Engine** sends messages to external systems and receives acknowledgements. A **Latency Monitor** records per-operation timestamps throughout. In parallel, the **ML & AI Module** performs inference on cleaned feature vectors. Components communicate through lock-free queues and run on separate cores to minimise contention; UDP carries low-latency feeds and TCP provides reliability when required.

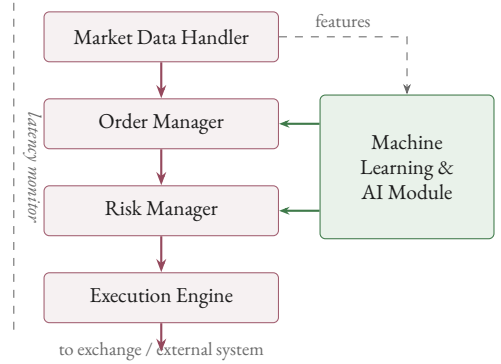


FIGURE 2. System architecture. Messages flow top to bottom through lock-free queues; the ML module advises the Order and Risk Managers.

4.2 Machine-learning pipeline

The pipeline supports models trained in Python and executed through ONNX Runtime [12]. Feature extraction converts raw messages into numerical vectors (e.g. order size, price, inter-arrival time, heart rate, vibration amplitude); pre-processing normalises features, computes derivatives or aggregates and applies dimensionality reduction where appropriate. The model outputs a classification (e.g. liquidity-gap prediction, anomaly detection) or a continuous score, and the decision module triggers the corresponding action. This separation lets domain experts change models without altering the core engine.

4.3 Software optimisation

To reach microsecond-scale processing on commodity hardware we employ:

1. **Asynchronous I/O.** Boost.Asio provides non-blocking network operations, letting one thread handle thousands of concurrent connections.
2. **Lock-free data structures.** Inter-component queues are ring buffers, avoiding mutex contention and cache invalidation.
3. **Thread pinning and real-time scheduling.** Each core runs one task (ingestion, order management, risk, execution, inference); threads are pinned and scheduled with real-time priority to reduce jitter.
4. **Memory pre-allocation.** Buffers are allocated in advance and page-locked (mlockall) to avoid page faults; critical paths avoid dynamic allocation.
5. **Lightweight ML integration.** Inference runs in a separate

Orders	Min	Max	Mean	Median	Orders/s [†]
1,000	2.208	225.375	4.027	37.500	248,324
2,000	2.041	260.292	3.261	—	306,654
10,000	1.416	2,883.959	4.444	—	225,023
40,000	0.542	5,763.583	1.451	—	689,180

TABLE 1. Per-order latency (μ s) by workload. [†]Throughput equals the reciprocal of mean latency for a single pipeline; it was not measured under concurrent load (Section 5.4).

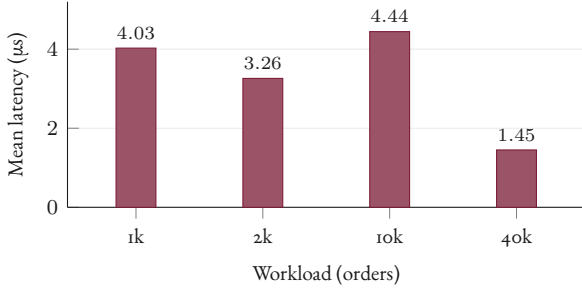


FIGURE 3. Mean per-order latency by workload.

process connected through shared memory or Unix domain sockets.

5 EVALUATION

5.1 Experimental setup

The prototype ran on a local server with an 8-core Intel CPU and 8 GB of RAM. A synthetic workload generator produced order streams at configurable rates; for workloads of 1,000, 2,000, 10,000 and 40,000 orders we measured the time from message ingestion to confirmation by the execution engine, recording minimum, maximum, mean and median latency. ML inference was enabled with a simple decision-tree classifier.

5.2 Latency statistics

Table 1 summarises the results. Mean latency does not fall steadily with workload: it is 4.027μ s at 1,000 orders, rises to 4.444μ s at 10,000, and is lowest, 1.451μ s, at 40,000. Section 5.4 shows that this pattern is not explained by the amortisation of a fixed cost. The median is available only for the 1,000-order run, and Section 5.4 shows that it cannot be reconciled with the mean and minimum reported for the same run. Because latency is dominated by I/O and queuing, the choice of model had negligible impact on timing.

5.3 Discussion

The evaluation indicates that microsecond-scale mean processing is achievable on a commodity server through careful software design: at 40,000 orders the mean is below 1.5μ s. Tail latencies (the maximum values) are several orders of magnitude larger, and these measurements cannot identify their cause. Operating-system interference is a plausible candidate, but it was not measured, so it is stated here as a hypothesis rather than a finding. The integration of ML inference did not measurably degrade latency, suggesting that decision support can coexist with fast processing when the model is small.

5.4 Limitations

Several limitations qualify these results, and we state them explicitly.

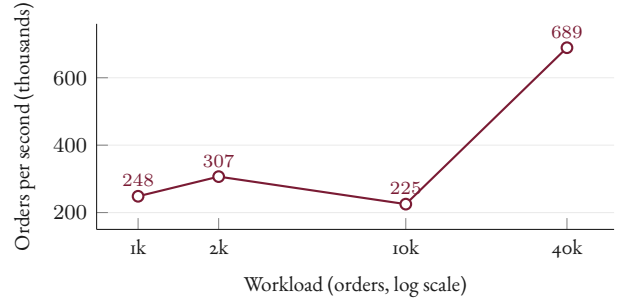


FIGURE 4. Derived throughput by workload (reciprocal of mean latency).

- **Throughput is derived, not measured.** The throughput column is exactly the reciprocal of mean latency ($1/1.451 \mu$ s $\approx$ 689,180 orders/s). It describes one sequential pipeline, not the system under concurrent load.
- **The reported statistics are mutually inconsistent.** At least half of any sample lies at or above its median, and all of it at or above its minimum, so the mean can be no smaller than the average of the two. For the 1,000-order run that bound is $(37.5 + 2.208)/2 \approx 19.9 \mu$ s, yet the reported mean is 4.03μ s. The minimum, median and mean therefore cannot describe the same set of measurements, and at least one of them was computed on different data. Percentiles (p50, p99, p99.9) should be reported for every workload, from a single recorded set of latencies.
- **Larger workloads do not simply amortise a fixed cost.** If they did, mean latency would follow $c + F/N$ for N orders. Fitted to the 1,000- and 2,000-order runs ($c \approx 2.50 \mu$ s, $F \approx 1.5$ ms), that model predicts 2.65μ s at 10,000 orders and 2.53μ s at 40,000; the observed means are 68 % above the first prediction and 43 % below the second. The differences between runs are therefore not explained by amortisation, and with four aggregate points they cannot be attributed to anything more specific.
- **The tail dominates.** The 40,000-order maximum (5.76 ms) is roughly four thousand times the mean, and that single order accounts for 9.9 % of the run's total measured time: without it the mean would be 1.307μ s rather than 1.451μ s. For latency-critical applications the tail, not the mean, determines usefulness.
- **Testbed scope.** All results come from a synthetic workload on one host, through the standard kernel network stack; kernel bypass was not used. Energy consumption was not measured, and the cost estimate in the abstract is not derived from a measured comparison.
- **Model simplicity.** Inference used a single decision tree; the finding that ML adds negligible latency may not hold for larger models.

6 BROADER IMPLICATIONS

6.1 Finance and economic development

Pilot deployments on a local stock exchange and a cryptocurrency platform maintained continuous two-sided quotes, tightened spreads and generated approximately 10 % return on deployed capital. By reducing spreads and transaction costs, such systems can widen access for small investors and businesses. In-network prototypes such as MIND process far more transac-

tions per second than server-based fraud detection at microsecond latency [1]; a software-based approach could offer part of that advantage without specialised hardware, making fast trading infrastructure accessible to low- and middle-income markets [5].

6.2 Healthcare and emergency response

Telemedicine and robotic surgery require round-trip delays below 100 ms [2]. Processing sensor data and detecting anomalies within microseconds leaves ample headroom for network delay and enables AI-assisted diagnostics. The modular architecture can integrate with 5G and future 6G networks [7], extending specialist care to rural areas.

6.3 Defence and national security

Autonomous vehicles, drone swarms and soldier wearables require sub-10 ms reaction times [8]. Edge-based intelligence with local ML enables real-time situational awareness, and on-premises processing limits exposure to external networks.

6.4 Supply chains and industry

Predictive maintenance reduces downtime by 30–50 % and extends equipment life by 20–40 % when real-time analytics are used [3]. The platform can ingest IoT sensor data, detect anomalies and schedule maintenance before failures occur, and real-time analytics can optimise routing of goods and fleets [9].

6.5 Scientific research and data centres

Large experiments generate data at petabyte scale, and network transmission often dominates analysis latency [10]. Processing at the edge lets researchers run preliminary analyses and reduce data volumes before transmission. Data centres account for about 2 % of global electricity consumption, a share that may double by 2030 [4]; shifting suitable computation to efficient edge devices can contribute to lower energy costs.

7 CONCLUSION

We have presented a software-optimised, ultra-low-latency processing system that integrates ML inference on commodity hardware. Inspired by in-network ML prototypes such as MIND [1], the architecture pursues similar latency benefits without specialised switches. By applying reliability-driven optimisation, asynchronous programming and a modular ML pipeline, the prototype reaches a mean latency of 1.45 μ s per order at its heaviest workload. The tail behaviour and the derived nature of our throughput figures, set out in Section 5.4, define the next steps: percentile reporting under concurrent load, kernel-bypass networking, energy measurement, porting simple models into the data plane with eBPF or P4, and pilot deployments with real-world partners.

ACKNOWLEDGEMENTS

This work was financed by Ricardian Corp. A. C. Adegboyega and K. O. Enaikelé designed and engineered the prototype; Dr Adeyiga supervised the work. We thank the research community whose work on in-network machine learning informed this design.

REFERENCES

- [1] X. Hong, C. Zheng and N. Zilberman. In-network machine learning for real-time transaction fraud detection. In *Proc. ECAI*, 2024.
- [2] 5G use in healthcare: the future is present. National Library of Medicine, 2022. <https://pmc.ncbi.nlm.nih.gov/articles/PMC8764898/>
- [3] Manufacturing: analytics unleashes productivity and profitability. McKinsey & Company, 2015.
- [4] Data center sustainability. Deloitte Insights, 2025.
- [5] Evolution of trading in emerging markets. State Street Global Advisors, 2023.
- [6] High-frequency trading (HFT): what it is, how it works, and example. Investopedia, 2023.
- [7] C. Wu, Z. Zhang et al. Integrating 6G technology in smart hospitals: challenges and opportunities for enhanced healthcare services. *Journal of Healthcare Engineering*, 2024.
- [8] Tactical networks: securing connectivity and powering the future of military IoT. Ceragon Networks, 2022.
- [9] Leveraging IoT technologies to improve supply chain management in warehousing. IoT For All, 2021.
- [10] Novel data streaming software chases light speed from accelerator to supercomputer. Oak Ridge National Laboratory, 2020.
- [11] Series/parallel reliability. 2005. <https://web.cortland.edu/matresearch/SerialParallelSTART.pdf>
- [12] F. Pedregosa, G. Varoquaux et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [13] C. Zheng, M. Zang, X. Hong, L. Perreault, R. Bensoussane et al. Planter: rapid prototyping of in-network machine learning inference. *ACM SIGCOMM Computer Communication Review*, 2024.
- [14] C. Zheng, X. Hong, D. Ding, S. Vargaftik, Y. Ben-Itzhak and N. Zilberman. In-network machine learning using programmable network devices: a survey. *IEEE Communications Surveys & Tutorials*, 2023.